



Tecnológico de Monterrey

Reporte técnico

Bum Soo Jang | A01665654

Pedro Luis Castañeda Pastelín | A01736619

Isaac Martínez Trujillo | A01735069

Joey Dominick Alcocer Hanson | A01784773

Materia - Desarrollo e implantación de sistemas de software

Profesores

.

Fecha de entrega:

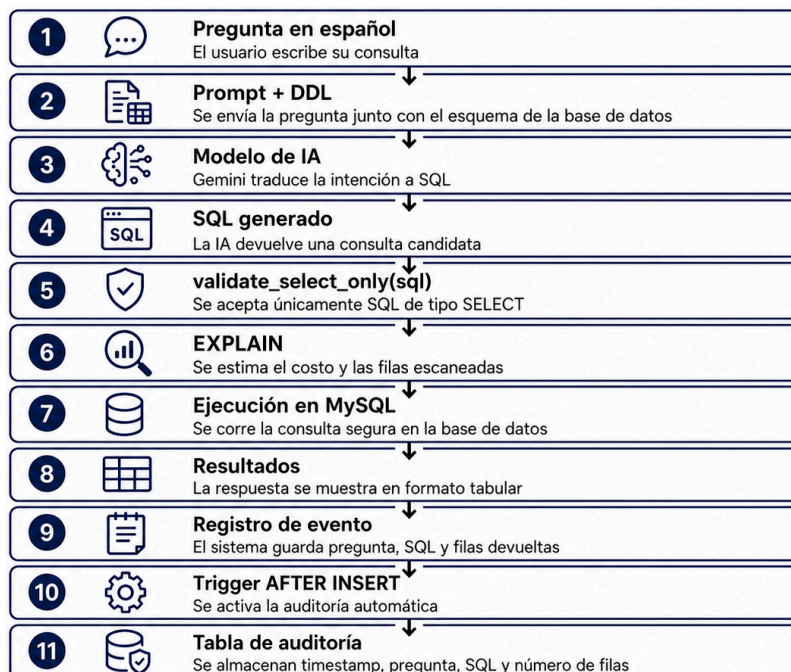
19 de mayo de 2026

1. Objetivo del proyecto

El objetivo de este proyecto es construir un asistente conversacional capaz de consultar una base de datos mediante preguntas escritas en lenguaje natural en español. La solución permite que un usuario no técnico, como un gerente, analista u operador, pueda realizar preguntas sobre el negocio sin escribir directamente consultas SQL.

2. Pipeline del sistema

El sistema se diseñó como un asistente *Text-to-SQL*, cuyo objetivo es convertir preguntas en español en consultas SQL ejecutables sobre una base de datos MySQL. El usuario escribe una consulta en lenguaje natural, el modelo de IA la traduce a SQL y el sistema aplica controles antes de ejecutarla. El pipeline es el siguiente:



3. Herramientas, prompt y seguridad

Para desarrollar el asistente se utilizaron las siguientes herramientas: **Python 3.11+**, **MySQL 8.0**, Gemini API (**gemini-2.0-flash**), **mysql-connector-python**, **tabulate**, **python-dotenv**. Las

credenciales de conexión y la API key no se escribieron directamente en el código. En su lugar, se utilizaron variables de entorno dentro de un archivo `.env`.

El prompt fue una parte clave del proyecto, ya que el modelo necesitaba conocer la estructura real de la base de datos para generar consultas correctas. Para ello, se incluyó el **DDL** completo de las tablas, instrucciones estrictas de salida y ejemplos de conversión de pregunta a SQL.

El uso de ejemplos *few-shot* ayuda a que el modelo respetara mejor la estructura de las tablas y generara consultas más cercanas a lo esperado.

Una de las partes más importantes del proyecto fue la implementación de la función `validate_select_only(sql)`. Esta capa evita que el modelo genere o ejecute instrucciones peligrosas sobre la base de datos.

La función valida que la consulta: inicie con *SELECT*, no contenga sentencias como *DROP*, *INSERT*, *UPDATE*, *DELETE*, *ALTER* o *TRUNCATE*, no incluya múltiples sentencias separadas por `;` y no intente modificar la estructura ni los datos de la base de datos.

4. Auditoría y pruebas realizadas

Para cumplir con el requisito de auditoría, se creó una tabla dedicada a registrar las consultas realizadas por el asistente. Esta tabla almacena la pregunta original del usuario, la consulta SQL generada, la fecha y hora de ejecución y el número de filas devueltas.

Se probaron al menos cuatro preguntas distintas relacionadas con la base de datos de e-commerce. Las pruebas evaluaron si el modelo generaba SQL válido, si las consultas se ejecutaban correctamente y si los resultados responden a la intención original del usuario.

#	Pregunta probada	Resultada esperado	Estado
1	¿Cuáles son los 5 clientes con mayor gasto total en los últimos 6 meses?	Lista de clientes ordenados por gasto total.	Funciona
2	¿Cuántos productos tienen stock por debajo del mínimo en este momento?	Conteo de productos con bajo inventario.	Funciona
3	¿Qué ciudad genera el mayor promedio de gasto por pedido?	Ciudad con mayor promedio de compra.	Funciona
4	Lista los pedidos del último mes que siguen pendientes, de mayor a menor monto.	Pedidos pendientes ordenados por monto.	Funciona

5. Caso de falla del modelo y corrección

Durante el desarrollo, se identificó un caso en el que el modelo generó una consulta incorrecta porque asumió el nombre de una columna que no existía en el esquema de la base de datos.

Pregunta original: ¿Cuáles son los 5 productos más vendidos este mes?

Problema detectado: El modelo generó una consulta usando una columna llamada *fecha_venta*, pero en la base de datos la fecha real estaba almacenada en la tabla de pedidos como *fecha_pedido*.

Causa: El prompt no indicaba con suficiente claridad qué tabla contenía la fecha de la venta ni cómo se relacionaban los pedidos con sus detalles.

Corrección aplicada: Se mejoró el prompt agregando el **DDL** completo, aclarando las relaciones entre tablas y añadiendo un ejemplo *few-shot* con una consulta que unía pedidos, *detalle_pedido* y *productos*.

Resultado: Después de ajustar el prompt, el modelo generó una consulta correcta usando las columnas reales de la base de datos. Este caso mostró que la calidad del SQL generado depende directamente de la precisión del contexto entregado al modelo.

6. Resultados y conclusión

El asistente logró traducir preguntas en español a consultas SQL ejecutables sobre MySQL. La solución permitió consultar información relevante de la base de datos, como productos con bajo inventario, ventas registradas, movimientos de inventario y alertas de reposición.

Los elementos más importantes del sistema fueron la validación *validate_select_only*, el uso de *EXPLAIN* y el registro de auditoría mediante trigger. Estas capas hacen que el asistente no solo funcione, sino que también sea más seguro, trazable y controlado.

En conclusión, el proyecto demostró cómo la inteligencia artificial puede integrarse con bases de datos para facilitar el acceso a información sin requerir conocimientos técnicos de SQL. Sin embargo, también mostró que un modelo de IA no debe conectarse directamente a una base de datos sin controles. Por ello, la validación de consultas, la estimación de costo y la auditoría son componentes indispensables para reducir riesgos y mantener evidencia de cada operación realizada.